Phase 2 | Updated Codebase Teaching Series

FastAPI edition, audited against the current repository snapshot.

Reading route

This source-course follows the current execution path, from ASGI startup to browser, model, schedule logic, and graph mapping. Use the nine Phase PDFs for interview framing and the volumes below for implementation-level study. Keep source files open as you read; names and behavior below refer to current code, not the older archived revision.

Volumes

1. **Startup, FastAPI registration, and page routes**: `run.py`, `app/__init__.py`, route imports, templates and static mounting.

2. **The prediction endpoint and JSON contract**: request parsing, date/time errors, threadpool, service closure, response sections and line filtering.

3. **Forecasting and model training**: artifact load, feature schema, hourly profile, synthetic generation, chronological split, leakage controls and metrics.

4. **Calendar, operating window, cleaning and synthetic data**: event provenance, hours, generated rows, typologies, and assumptions.

5. **Service scheduling, mapping and shortest paths**: demand shares, headway/capacity heuristic, canonical-name mapping, line-expanded nodes, Dijkstra.

6. **Frontend templates and JavaScript**: initial template values, interaction events, async fetch, race control, charts, search, themes.

7. **Styles, deployment and legacy boundaries**: CSS cascade, requirements, Procfile, environment and files that are not runtime inputs.

8. **End-to-end execution trace**: startup once, each request, closed-time branch, future scenario, API response, and route calculation.

How to study each volume

For each module ask: what runs; when does it run; what are its inputs and outputs; which assumptions are hidden; what can fail; what would I test? The included exercises are reasoning exercises, not claims that a test suite currently exists.

Snapshot-level source map

- HTTP app and pages: `app/__init__.py`, `app/routes.py`, `run.py`, `Procfile`.
- ML: `app/forecasting.py`; offline workflow `pipeline/01_audit_and_clean_data.py`, `pipeline/02_generate_calibrated_dataset.py`, `pipeline/03_train_and_evaluate_forecaster.py`.
- Calendar/service: `app/calendar_context.py`, `app/operating_hours.py`.
- Schedule and line split: `app/scheduling_optimizer.py`, `data/cleaned/metro_line_operations.csv`, `data/synthetic/line_distribution_model.json`.
- Network: `app/network_topology.py`, `app/travel_graph.py`, `data/cleaned/stations_cleaned.csv`.
- Browser: `app/templates/`, `app/static/js/`, `app/static/css/`.
- Fitted assets/metrics: `models/footfall_forecaster.joblib`, `models/forecaster_metadata.joblib`, `models/evaluation_metrics.json`.

Current repository snapshot

The cleaned station table has 243 records and the route topology defines 11 project route keys. All 243 topology labels map to canonical station records in this checkout; shortest times from Rajiv Chowk return 242 destinations and omit the origin. Synthetic volume is 243 x 728 x 19 = 3,361,176 records. The chronological holdout from 2026-10-01 has 438,615 rows. HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, MAPE 5.18% on synthetic labels.

Key model and data caution

The generated labels encode scenario assumptions. A chronological holdout tests generalization to later generated dates, not validity on actual passengers. State this before quoting metrics. A synthetic model that scores well can be faithfully learning the simulator rather than Delhi passenger behavior. Capacity/headway and route-time values are project assumptions/estimates. No live ridership, train dispatch, database, or authentication is present.